

TRABAJO PRÁCTICO: Operaciones Escritura en API REST (POST, PUT, DELETE)

Objetivo

- a) Diseñar e implementar los **DTOs de entrada** (Request DTOs) para desacoplar el contrato de la API del modelo de base de datos en operaciones de modificación.
- b) Implementar la lógica de negocio necesaria en la capa de servicios (@Service) para la creación en cascada, actualización y eliminación/anulación de facturas de venta.
- c) Exponer los endpoints REST correspondientes mediante @PostMapping, @PutMapping y @DeleteMapping, respetando las convenciones HTTP y el manejo adecuado de códigos de respuesta (201 Created, 200 OK, 204 No Content, 404 Not Found).

1- DTOs de Entrada (Request DTOs)

Consignas

- a) Crear las clases FacturaRequestDTO y FacturaDetalleRequestDTO para estructurar los datos enviados por los clientes en las peticiones **POST** y **PUT**.
- b) Los DTOs de entrada deben recibir **identificadores (IDs)** para vincular las relaciones (clienteId, condicionIvaId, puntoVentaId, tipoMonedaId, articuloId), evitando enviar objetos entidad anidados en la petición JSON.

2- Lógica de Negocio en la Capa de Servicio (FacturaService)

Consignas

- a) Incorporar la anotación @Transactional para garantizar la atomicidad de las transacciones de escritura.
- b) **Creación (crear):**
 - Buscar las entidades asociadas por ID (PuntoVenta, CondicionIva, TipoMoneda, Cliente, ListaPrecioArticulo). Lanzar una excepción si no existen.
 - Mapear el FacturaRequestDTO a la entidad FacturaVenta y calcular automáticamente los importes netos, IVA y total según los detalles proporcionados.
 - Guardar la factura y verificar que sus detalles se persistan en cascada.
- c) **Actualización (actualizar):**
 - Buscar la factura por su ID. Si no existe, lanzar excepción de recurso no encontrado.
 - Actualizar cabecera e ítems.
- d) **Eliminación / Anulación (eliminar):**
 - Implementar la eliminación física de la factura o una anulación lógica (cambio de estado = "ANULADA" y asignación de fechaAnulacion = new Date()), según corresponda a las reglas de negocio del sistema de facturación.

3- Capa de Controladores REST (FacturaRestController)

Consignas

1. Mapear los siguientes endpoints en el controlador existente (/api/facturas):
 - **POST /api/facturas**: Recibe FacturaRequestDTO, invoca al servicio para crear la factura y retorna el código de estado 201 Created junto con el FacturaReporteDTO generado.
 - **PUT /api/facturas/{id}**: Recibe el identificador por path variable y los datos/estado a actualizar en el cuerpo de la petición. Retorna 200 OK.
 - **DELETE /api/facturas/{id}**: Elimina la factura especificada por ID y retorna la respuesta 204 No Content.

4- Frontend CRUD de Facturación con TypeScript y HTML

Objetivo

- a) Modelar los tipos de datos en el frontend creando **Interfaces de TypeScript** que espejen exactamente la estructura de los **DTOs de Spring Boot** (FacturaReporteDTO, FacturaRequestDTO, FacturaDetalleRequestDTO).
- b) Implementar una capa de cliente HTTP (FacturaService.ts) mediante async/await y la API fetch para consumir los endpoints REST (GET, POST, PUT, DELETE).
- c) Diseñar la interfaz gráfica en **HTML5** y vincularla mediante **TypeScript** para permitir la carga dinámica de ítems, el filtrado de comprobantes, el alta de facturas y las acciones de anulación/eliminación.

5- Definición de Interfaces TypeScript (Espejo de DTOs)

Consignas

- a) Crear el archivo models/factura.dto.ts.
- b) Definir los tipos e interfaces asegurando la correspondencia exacta de nombres de campos y tipos de datos con los DTOs de Java.

6- Capa de Servicio API (FacturaService.ts)

Consignas

- a) Crear la clase FacturaService encargada de centralizar las llamadas HTTP al backend (http://localhost:8080/api/facturas).
- b) Implementar los métodos:
 - obtenerFacturas(filtros: FacturaFiltrosDTO): Promise<FacturaReporteDTO[]>
 - crearFactura(factura: FacturaRequestDTO): Promise<FacturaReporteDTO>
 - actualizarEstado(id: number, estado: string, observaciones?: string): Promise<FacturaReporteDTO>

- `eliminarFactura(id: number): Promise<void>`

7- Interfaz de Usuario en HTML5 y Lógica UI en TypeScript

Consignas

- a) Diseñar el archivo `index.html` compuesto por tres secciones principales:
 - Formulario de búsqueda con filtros.
 - Formulario de alta de factura con tabla dinámica para cargar múltiples renglones/detalles.
 - Tabla con el listado de comprobantes registrados e interacciones por fila (anular y eliminar).
- b) Crear `app.ts` para manipular el DOM, gestionar el estado de los ítems en memoria y conectar los eventos con el servicio HTTP.